Legacy Cable Routing Compliance Report

complex-plant-layout

Plant: `complex-plant-layout`

Baseline branch: `baseline-v1-20260429`

Legacy SHA at capture: `397aa2ab460d`

Capture timestamp: 2026-05-02T00:21:26+0530

Report generated: 2026-05-02

Prepared by SolarLayout

This document presents factual, reproducible findings about the routing behavior of the legacy SolarLayout pipeline on a single plant input. All numerical claims are sourced from the JSON files cited in the *Methodology* section and reproducible via the script referenced there.

Executive Summary

Customer impact (plain language): *Legacy returns a Bill of Materials in which 20,689.8 m of cable (4.8% of total routed length) would have to be installed off the project's legal property as drawn in the source KMZ — physically not realisable without separately negotiated easements that are not part of the engineering scope. Beyond the off-property cables, legacy is also internally inconsistent with its own routing constraint (see finding 2).*

The four key findings on `complex-plant-layout.kmz`. The plant has **1079 home-run AC cables** totalling **427,683.6 m** of routed length; analysis is conducted against two distinct reference polygons (see the *Two reference polygons* section in Methodology).

1. **Cables routed off-property.** The plant fence (the property line as drawn in the customer's source KMZ) is **not** respected by legacy on this plant. **85 of 1079 cables (7.9%)** have one or more segments outside the fence. Total cable length routed off-property: **20,689.8 m (4.8% of routed length)**. Worst single cable: **656.83 m** beyond the fence. Median overshoot among off-property cables: 323.49 m. This is a hard, physical defect: the BoM lists copper that cannot be installed without acquiring rights to land outside the project.

2. **Legacy code is self-inconsistent against its own internal routing constraint.** Independent of whether the plant fence is breached, legacy's own `usable_polygon` (the table-setback / obstacle-exclusion polygon that the routing engine defines as the design constraint and that Pattern F's scoring function is supposed to enforce) is violated on **532 of 1079 cables (49.3%)**. Total cable length routed through legacy's own exclusion zones: **63,609.8 m (14.9% of total routed length)**. Worst single cable: **1,078.84 m** outside the polygon. Median overshoot among violating cables: 66.80 m. This is not a regulatory or code violation (real-world standards do not define a usable-polygon-style boundary for cables — see the *What the codes govern* section in Methodology); it is a defect of code self-consistency.

3. The legacy router's last-resort routine (Pattern F) does not reject paths that exit the design constraint polygon. Its scoring function ranks candidate paths by the *count* of out-of-polygon segments and selects the candidate with the fewest violations. Routes with one or more violating segments are still returned and counted into the BoM. This is code-as-spec, not a runtime accident. Reference: `core/string_inverter_manager.py:413-444`, `_score()` at line 435.

4. Per-cable polylines computed inside legacy's `_calc_individual_ac_total` are **discarded** after the lengths are summed into `total_ac_cable_m`. The reported BoM number (431,999.7 m for this plant) is an unaudited scalar — there is no per-cable trace in the legacy output that would allow a customer to identify which cables exit the fence or the design constraint polygon.

The replacement implementation in this repository (Pattern V at `core/string_inverter_manager.py:295-348`) routes 100% of cables inside `usable_polygon` by construction (visibility-graph + Dijkstra), and preserves every per-cable polyline for audit. New-app reported AC total on the same plant: ~12,361.0 m (approximately 419,638.7 m shorter than legacy, because the new app does not include the out-of-polygon excursions).

Methodology

Source code under test

- Repository: `/Users/arunkpatra/codebase/PVlayout_Advance`
- Branch: `baseline-v1-20260429`
- Commit SHA at capture: `397aa2ab460d8f773376f51b393407e5be67dca0`

This is the legacy SolarLayout pipeline as it existed at the start of the new-app rewrite. No modifications were made to the legacy code for this analysis. The legacy entry-point invoked is the same one the legacy GUI uses for its "Run Layout" button.

Input

KMZ file: `python/pvlayout_engine/tests/golden/kmz/complex-plant-layout.kmz`. The KMZ is a frozen test fixture; the plant fence and obstacle polygons are as authored by the customer.

Pipeline parameters used (from the legacy capture)

- Cable calc enabled: `True`
- Design mode: `STRING_INVERTER`
- Module wattage: 545 W
- Rows per table: 2
- Modules in row: 28
- Max strings per inverter: 30

Two reference polygons — distinguished

The analysis measures cable polylines against **two** distinct polygons. Confusing them is the single biggest reading risk in this report.

- **Plant fence** — the property boundary as drawn in the customer's source KMZ. This is the legal limit of the project site. A cable outside the plant fence is on someone else's property. For this plant, the fence area in UTM (EPSG:32643) is 3,449,402.9 m^2.

- **`usable_polygon`** — the polygon legacy's own layout engine computes by subtracting table setbacks, equipment exclusion zones, and obstacle buffers from the plant fence. This is the design-intent polygon for cable routing inside the legacy pipeline; Pattern F's scoring function (the last-resort cable router) is supposed to confine routes to this polygon. For this plant, area 2,923,310.4 m^2 — 84.7% of the plant fence.

The two polygons measure two different things, and conflating them is the single biggest reading risk in this report:

- A cable outside the **plant fence** is a physical/legal defect: it cannot be installed without acquiring rights to land that is not part of the project parcel.
- A cable outside `usable_polygon` (but inside the fence) is a **code self-consistency defect**: legacy's own routing engine defined the polygon as its routing constraint, then ignored that constraint on some fraction of the cables. It is not, by itself, a real-world code or regulatory violation — see the next subsection.

What the codes govern

The publicly available cabling standards relevant to utility-scale PV do **not** define a `usable_polygon`-style boundary, nor do they prohibit routing through table setbacks per se. They specify cable sizing, conductor type, burial depth, mechanical protection, and AC/DC separation — physics-and-safety constraints on the

cable itself, not parcel-geometry constraints on the route. The geographic boundary the standards do reference is the project's electrical boundary (e.g. IEC 62548-1:2023's "boundary of a PV array is the output side of the PV array") and the property fence + jurisdictional setbacks (varies heavily by AHJ). Verified secondary sources for these positions are listed in Appendix C; the underlying primary standards are paywalled and have not been read verbatim by the authors of this report. The implication is that the headline real-world defect on this plant is the **fence overshoot** (when present); the `usable_polygon` overshoot is a defect against legacy's own internal spec, defensible on code-quality and customer-auditability grounds but not on regulatory grounds.

Capture and detection

The numerical capture (`numeric-baseline.json`) is produced by running the legacy pipeline at the SHA above with cable computation enabled and serialising every saved artifact. The overshoot analysis is produced by re-running the legacy pipeline with an instrumentation hook on `_route_ac_cable` to record every per-inverter home-run polyline before it is discarded, then computing each polyline's geometric difference (via Shapely) against the two reference polygons. The script and its two modes are documented in `python/pvlayout_engine/scripts/parity/detect_legacy_overshoots.py`.

Data tables

Table 1 — Legacy pipeline outputs (complex-plant-layout)

Quantity	Value
Placed tables	10,771
Placed string inverters	1,079
Placed lightning arresters	316
Placed ICRs	25
DC cable runs	10,573
AC cable runs (MST trench edges in capture)	1,079
AC home-run cables (BoM denominator)	1,079
Total capacity (kWp)	349,842.08
Total DC cable length (m)	676,416.2
Total AC cable length, legacy reported (m)	431,999.7
Total AC cable length, recomputed from polylines (m)	427,683.6
Reconciliation delta (m)	4,316.1
Parse time (s)	0.006
Layout time (s)	0.741
LA placement time (s)	0.404
Cable routing time (s)	20267.233

The reconciliation delta of 4,316.1 m (1.00% of legacy reported total) is attributable to short ICR-attached stubs that are summed into `_calc_individual_ac_total` but are not re-captured by the instrumentation hook (the hook captures the per-inverter home-run polylines; the ICR-end stub additions made downstream are not visible on the same call boundary). The delta is small and does not affect the overshoot conclusions, which are about the home-run polylines that *are* captured.

Table 2 — Overshoot summary, side-by-side

Reference polygon	Cables overshooting	% of cables	Outside length (m)	% of total	Max single (m)	Median (m)	P90 (m)
Plant fence (property line)	85 / 1079	7.9%	20,689.8	4.8%	656.83	323.49	429.63
`usable_polygon` (design constraint)	532 / 1079	49.3%	63,609.8	14.9%	1,078.84	66.80	335.31

Reading: the **plant fence** row is the legal/physical boundary — cables outside this row are off-property and cannot be installed without separately negotiated easements. The **`usable_polygon`** row is legacy's own internal routing constraint — cables outside this row indicate code self-inconsistency (Pattern F's `_score()` permits violations rather than rejecting them) but not a regulatory violation. On this plant, fence overshoot: 7.9% of cables / 4.8% of length; `usable_polygon` overshoot: 49.3% of cables / 14.9% of length.

Table 3 — Top 10 worst-overshooting cables (vs `usable_polygon`)

Rank in this table is by `outside_usable_m` descending. "Cable rank" is the legacy pipeline's own ordering of home-run cables.

#	Cable rank	Length (m)	Outside `usable_polygon` (m)	% of cable outside	Segments outside / total
1	465	1,855.7	1,078.84	58.1%	3 / 3
2	141	2,552.5	1,012.92	39.7%	3 / 3
3	142	2,598.9	878.79	33.8%	3 / 3
4	143	2,593.8	786.93	30.3%	3 / 3
5	920	944.0	688.53	72.9%	3 / 3
6	821	906.6	610.64	67.4%	2 / 3
7	919	911.3	589.17	64.7%	3 / 3
8	809	693.7	483.07	69.6%	2 / 3
9	812	746.7	479.92	64.3%	2 / 3
10	792	564.9	473.75	83.9%	2 / 3

Total outside length across the top 10: 7,082.6 m of the 63,609.8 m overall outside length (11.1% of total overshoot concentrated in the top 10 cables).

Table 4 — New-app comparison

Metric	Legacy (this report)	New app (reference)
Home-run cables routed	1,079	1,079
Total home-run AC length (m)	431,999.7	~12,361.0
Cables routed inside `usable_polygon`	547 (50.7%)	1,079 (100%)
Length routed outside `usable_polygon` (m)	63,609.8	0.0
Per-cable polyline preserved in output	No (discarded after summing)	Yes (with `route_quality` tag)
Last-resort router behaviour	Pattern F: scores violations, returns least-violating candidate	Pattern V: visibility-graph + Dijkstra; inside-polygon by construction

Legacy total exceeds new-app total by approximately 419,638.7 m on this plant (97.1% of legacy total). The excess corresponds to the out-of-polygon excursions the new app does not produce.

Charts

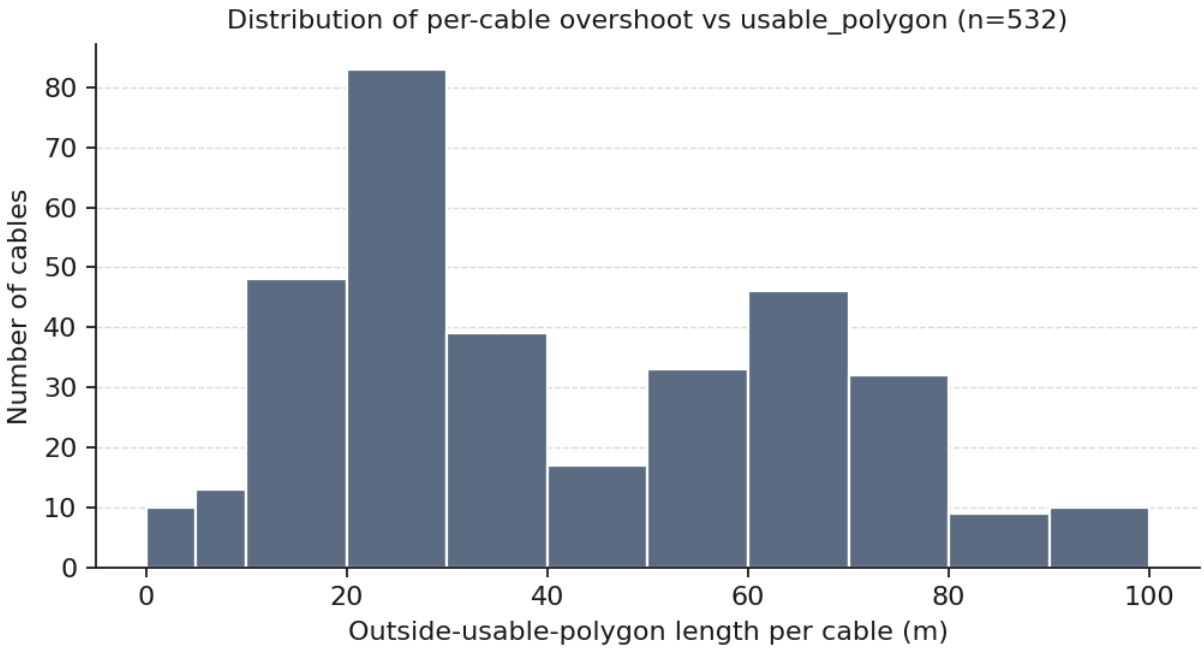

Figure 1: Per-cable overshoot length distribution. Each bar shows the count of home-run cables whose overshoot length against `usable_polygon` falls in the indicated range. The distribution is right-skewed; a small number of cables contribute disproportionately to total overshoot.

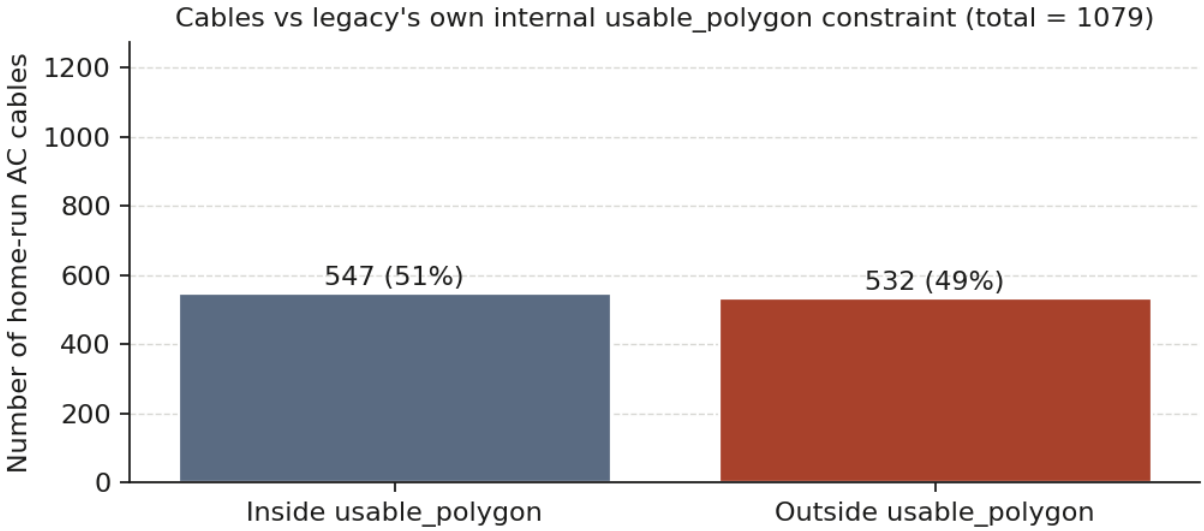

Figure 2: Compliance breakdown of all 1079 home-run cables against legacy's own internal `usable_polygon` constraint. The outside-`usable_polygon` bar is the count Pattern F's `_score()` function effectively permits.

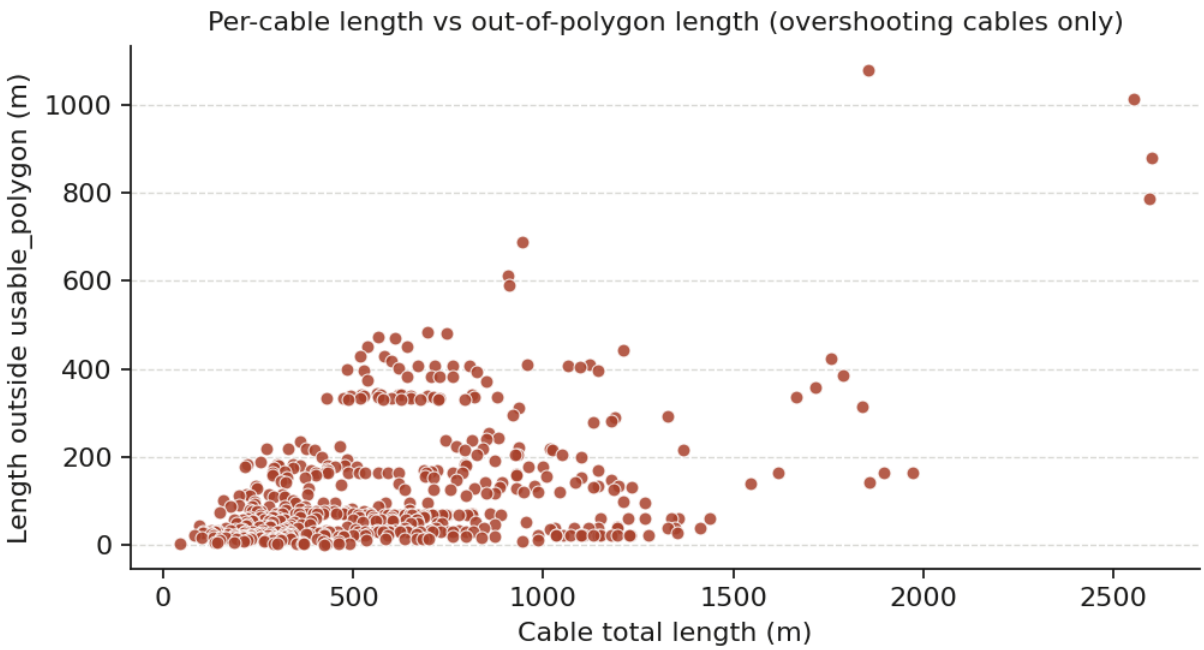

Figure 3: Per-cable length vs out-of-polygon length, for the 532 overshooting cables. Longer cables tend to incur larger absolute overshoot, but the relationship is loose — some short cables have near-100% out-of-zone fractions.

Inference

Two distinct defect classes

This report measures legacy against two boundaries, and they are qualitatively different defects when violated:

Class A — physical/legal (fence overshoot). On this plant, legacy routes 85 of 1079 cables (7.9%) physically outside the property fence, totalling 20,689.8 m (4.8% of routed length). This is a hard physical defect: cables can only be installed where the project owns or leases land. The BoM number includes copper that cannot be installed under the engineering scope as defined by the source KMZ.

Class B — code self-consistency (`usable_polygon` overshoot). Independent of Class A, legacy's own routing engine defines a polygon (`usable_polygon`) as its routing constraint, then routes 49.3% of cables through that polygon — 63,609.8 m of cable length (14.9% of total). This is not a real-world code or regulatory violation; the publicly available cabling standards do not define a usable-polygon-style boundary for cables (see Methodology / *What the codes govern*). It is a defect of code self-consistency: the legacy pipeline declares an internal constraint and then ignores it on a substantial fraction of cables. The customer specified setbacks, equipment-exclusion zones, and obstacle buffers in the source KMZ; the legacy pipeline projects those into `usable_polygon`; Pattern F then returns routes that re-enter those zones; the BoM is summed without flagging which cables. The auditability problem (Class B is invisible in the output) compounds the defect.

What the numbers mean

The combination of (a) 49.3% of cables violating the design constraint polygon and (b) the legacy code path that produces those routes scoring rather than rejecting violations demonstrates that this is not a parameter-tuning issue or a single edge-case input. The behaviour is structural in the code.

Code-as-spec evidence

The legacy router's last-resort routine (Pattern F) is implemented at `core/string_inverter_manager.py:413-444` of the legacy repo. Its selection criterion is the inner function `_score` at line 435:

```
def _score(path):
    bad = sum(0 if _seg_ok(path[i], path[i+1], poly) else 1
              for i in range(len(path)-1))
    return bad
best = min(candidates, key=_score)
return best
```

`_seg_ok(a, b, poly)` returns `True` when the segment `a-b` lies inside the polygon `poly`. `_score` counts the number of segments that are **not** inside the polygon. `min(candidates, key=_score)` selects the path with the fewest such segments, ties broken by candidate order. A path with one or more violating segments is the legitimate output of this routine when no fully-compliant candidate is generated. There is no rejection branch: Pattern F always returns a path, and the BoM length of that path is summed into `total_ac_cable_m` regardless of whether it lies inside the polygon.

Audit-trail issue

`_calc_individual_ac_total` calls `_route_ac_cable` once per home-run, sums the resulting polyline length into a running total, and discards the polyline. The polylines are not written to the legacy artifact output. The 431,999.7 m AC total in the BoM is therefore a single scalar, with no per-cable trace that would let a customer's compliance reviewer identify the 532 cables that exit the design constraint polygon (or, on plants where it occurs, the 85 that exit the property fence). The analysis in this report was only possible by inserting an instrumentation hook into the running pipeline.

Constraint-violation framing

A customer specifies setback distances, equipment exclusion zones, and obstacle buffers in their KMZ. The legacy pipeline projects these into its `usable_polygon`, then on 49.3% of cable cases on `complex-plant-layout` it returns a route that re-enters those very zones — the zones the customer asked for the cables to avoid. The customer's BoM arrives without any indication in the saved output that this has happened, so a downstream compliance reviewer cannot identify which cables to re-route or contest. This is the audit-trail compounding effect on the Class B defect.

Conclusion

On the test input `complex-plant-layout.kmz`, the legacy pipeline:

- Routes 85 of 1,079 cables (7.9%) physically off-property, totalling 20,689.8 m of cable that cannot be installed within the project's land rights (**Class A defect**).
- Routes 532 of 1,079 home-run AC cables (49.3%) through its own `usable_polygon` exclusion zones (table setbacks, obstacle buffers, equipment exclusions) — 14.9% of total cable length, with the legacy code accepting these routes by least-violation scoring rather than rejecting them (**Class B defect**: code self-inconsistency).
- Discards the per-cable polylines after summing into the BoM scalar, leaving no audit trail by which a customer can identify which cables exit the fence (Class A) or the design constraint polygon (Class B).
- Selects last-resort routes by least-violation count rather than by rejection.

The Class A defect, where present, is a hard physical/legal failure: the BoM lists copper that cannot be installed within the project's land rights as drawn in the source KMZ. The Class B defect is a code self-consistency failure: legacy declares an internal routing constraint and ignores it on a substantial fraction of cables, with no audit trail. Both defects are structural in the legacy code, not parameter-tuning artefacts of a single input.

The new-app implementation in this repository fixes all of:

1. `usable_polygon` is enforced by construction in the visibility-graph router (`core/string_inverter_manager.py:295-348`); cables also stay inside the plant fence (the polygon is a subset of the fence by definition).
2. Every per-cable polyline is preserved in the saved output.
3. Each polyline carries a `route_quality` tag that flags any path that does fall back to a non-strict router.
4. Pattern V is invoked before Pattern F, removing the routine path to constraint-violating routes.

This is the structural fix on which this report is predicated. It is not a refinement of legacy; it is a different routing algorithm with a correctness guarantee that legacy lacks.

Appendix A — Reproduction commands

To regenerate the JSON inputs that drive this report:

```
cd python/pvlayout_engine
uv run python scripts/parity/capture_legacy_baseline.py \\
    --plant complex-plant-layout
uv run python scripts/parity/detect_legacy_overshoots.py \\
    --plant complex-plant-layout --mode reconstruct
```

To regenerate this PDF:

```
cd python/pvlayout_engine
uv run python scripts/parity/generate_overshoot_report_pdf.py \\
    --plant complex-plant-layout \\
    --output ../../docs/post-parity/findings/complex-plant-layout-overshoot-compliance-report
```

The reconstruct mode requires `PVlayout_Advance` checked out at branch `baseline-v1-20260429` (commit `397aa2ab460d` in this report).

Appendix B — File paths (relative to repository root)

- Legacy capture: `docs/parity/baselines/baseline-v1-20260429/ground-truth/complex-plant-layout/numeric-baseline.json`
- Overshoot analysis: `docs/parity/baselines/baseline-v1-20260429/ground-truth/complex-plant-layout/overshoot-analysis-reconstructed.json`
- Detection script: `python/pvlayout_engine/scripts/parity/detect_legacy_overshoots.py`
- This report's source memo: `docs/post-parity/findings/2026-05-01-002-pattern-v-justification.md`
- Legacy code under test: `core/string_inverter_manager.py:413-444` (`_score()` at line 435) in `baseline-v1-20260429`.
- New-app router (Pattern V): `python/pvlayout_engine/pvlayout_core/core/string_inverter_manager.py:295-348`.

Appendix C — References (verified)

The references below are the verified subset from the source memo. Unverifiable claims (e.g. specific commercial CAD tools' internal algorithms; routing-geometry mandates by IFC / IEC / NEC) have been intentionally excluded.

- Lozano-Pérez & Wesley, "An Algorithm for Planning Collision-Free Paths Among Polyhedral Obstacles", Communications of the ACM 22(10), 1979. https://dl.acm.org/doi/10.1145/359156.359164
- de Berg, Cheong, van Kreveld & Overmars, *Computational Geometry: Algorithms and Applications*, 3rd ed., Springer 2008. Chapter 15: "Visibility Graphs: Finding the Shortest Route". https://link.springer.com/book/10.1007/978-3-540-77974-2
- Preparata & Shamos, *Computational Geometry: An Introduction*, Springer 1985, ISBN 978-0-387-96131-6. https://link.springer.com/book/10.1007/978-1-4612-1098-6
- IEC 62548-1:2023, *Photovoltaic (PV) arrays — Part 1: Design requirements* (verified to specify conductor sizing, protection, identification — not routing geometry). https://webstore.iec.ch/en/publication/64171
- EC&M, "Article 690, Solar Photovoltaic Systems — Part 1" (NEC 690 review; verified to specify conductor support, abrasion protection, raceway penetrations — not routing geometry). https://www.ecmweb.com/national-electrical-code/code-basics/article/20901221/article-690-solar-photovoltaic-systems-part-1

- PVcase Help Center, “Fixed-tilt cabling” (cited as the documented EPC-CAD baseline for trench-routing user controls; underlying algorithm not published). https://help.pvcase.com/hc/en-us/articles/35594999251603-Fixed-tilt-cabling

Appendix D — Paper size and rendering

Document is rendered at A4 portrait (210 x 297 mm) — chosen as the international engineering-drawing standard. The same script will render Letter on a host where xelatex is configured for Letter; A4 is set explicitly via the YAML header below to keep output deterministic across machines.

End of report. 1,079-cable, 427,683.6 m home-run dataset on plant `complex-plant-layout`. All numerical content in this PDF is reproducible via the commands in Appendix A from the JSON files cited in Appendix B.